AI ASSISTED CODING

ASSIGNMENT-21

T.Hima Bindhu

2303A52050

Batch:38

Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration

Task 1 – AI-Assisted Pair Programming

• Use AI to generate an initial program (e.g., a simple To-Do List Manager).

• Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).

• Compare the original and improved versions to understand the role of AI in collaboration.

Prompt:

Generate a simple To-Do List Manager program in Python with basic add, delete, and display features. Then suggest improvements like error handling and additional functionalities.

Code & Output:

```python
class TodoListManager:
    def __init__(self):
        self.tasks = []
    def add_task(self, task):
        self.tasks.append({'task': task, 'completed': False})
        print(f"Task '{task}' added.")
    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- Your To-Do List ---")
        for i, task in enumerate(self.tasks):
            status = "[DONE]" if task['completed'] else "[PENDING]"
            print(f"{i + 1}. {task['task']} {status}")
        print("-----------------------")
    def mark_task_complete(self, task_index):
        if 0 <= task_index < len(self.tasks):
            self.tasks[task_index]['completed'] = True
            print(f"Task '{self.tasks[task_index]['task']}' marked as complete.")
        else:
            print("Invalid task number.")
def main():
    manager = TodoListManager()
    while True:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task Complete")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            task = input("Enter the task: ")
            manager.add_task(task)
        elif choice == '2':
            manager.view_tasks()
        elif choice == '3':
            try:
                task_num = int(input("Enter the task number to mark as complete: ")) - 1
                manager.mark_task_complete(task_num)
            except ValueError:
                print("Invalid input. Please enter a number.")
        elif choice == '4':
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == '__main__':
    main()
```

```
1. Add Task
2. View Tasks
3. Mark Task Complete
4. Exit
Enter your choice: 1
Enter the task: Add task
Task 'Add task' added.

1. Add Task
2. View Tasks
3. Mark Task Complete
4. Exit
Enter your choice: 2

--- Your To-Do List ---
1. Add task [PENDING]
-----------------------

1. Add Task
2. View Tasks
3. Mark Task Complete
4. Exit
Enter your choice: 3
Enter the task number to mark as complete: 5
Invalid task number.

1. Add Task
2. View Tasks
3. Mark Task Complete
4. Exit
Enter your choice: 4
Exiting To-Do List Manager. Goodbye!
```

Explanation:

This task demonstrates how AI can act as a coding partner by generating an initial solution quickly. The student then interacts with AI to refine the code by adding features such as input validation, file storage, or a user-friendly interface. Comparing both versions helps understand how AI improves code

quality and efficiency. It also highlights collaborative development between human thinking and AI suggestions. This process builds better problem-solving and coding practices.

Task 2 – Version Control Integration

Initialize a Git repository for your project.

Perform the following steps:

• Create and switch to a new branch.

• Add AI-assisted modifications.

• Commit with a meaningful message.

• Merge changes back into the main branch.

Reflect on how version control supports collaborative development with AI assistance.

Prompt:

Guide me through initializing a Git repository, creating a new branch, making changes, committing them, and merging back into the main branch.

Code & Output:

```python
# Get the content of the TodoListManager cell directly
# This content needs to be manually copied or retrieved via Colab's API if available
# For this example, I will use the known content of cell 9ef2d407
code_to_save = """class TodoListManager:
    def __init__(self):
        self.tasks = []
    def add_task(self, task):
        self.tasks.append({'task': task, 'completed': False})
        print(f"Task '{task}' added.")
    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- Your To-Do List ---")
        for i, task in enumerate(self.tasks):
            status = "[DONE]" if task['completed'] else "[PENDING]"
            print(f"{i + 1}. {task['task']} {status}")
        print("-----------------------")
    def mark_task_complete(self, task_index):
        if 0 <= task_index < len(self.tasks):
            self.tasks[task_index]['completed'] = True
            print(f"Task '{self.tasks[task_index]['task']}' marked as complete.")
        else:
            print("Invalid task number.")
def main():
    manager = TodoListManager()
    while True:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task Complete")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            task = input("Enter the task: ")
            manager.add_task(task)
        elif choice == '2':
            manager.view_tasks()
        elif choice == '3':
            try:
                task_num = int(input("Enter the task number to mark as complete: ")) - 1
                manager.mark_task_complete(task_num)
            except ValueError:
                print("Invalid input. Please enter a number.")
        elif choice == '4':
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == '__main__':
    main()"""
# Save the code to a file
with open('todo_list_manager.py', 'w') as f:
    f.write(code_to_save)
print("To-Do List Manager code saved to 'todo_list_manager.py'")
```

```
To-Do List Manager code saved to 'todo_list_manager.py'
```

```
!git init
!git config --global user.email "colab-user@example.com"
!git config --global user.name "Colab User"
```

```
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /content/.git/
```

```
!git add todo_list_manager.py
!git commit -m "Initial commit of To-Do List Manager"
```

```
[feature/robust-input (root-commit) c1ae8e3] Initial commit of To-Do List Manager
 1 file changed, 49 insertions(+)
 create mode 100644 todo_list_manager.py
```

```
!git branch
!git checkout -b feature/robust-input
!git branch
```

```
Switched to a new branch 'feature/robust-input'
```

```
# Assuming master is currently at c1ae8e3 and todo_list_manager.py has uncommitted AI changes

# Step 1: Stash the current uncommitted changes (AI-assisted modifications)
# This is necessary because the file was modified on master after the previous merge attempt
!git stash save "AI-assisted modifications for robust input validation"

# Step 2: Create and switch to a new feature branch for the AI modifications
!git checkout -b feature/robust-input-v2

# Step 3: Apply the stashed changes to the new feature branch
!git stash pop

# Step 4: Commit the AI-assisted modifications on the feature branch
!git add todo_list_manager.py
!git commit -m "AI-assisted: Enhance input validation in main function (re-attempt)"

# Step 5: Switch back to master
!git checkout master

# Step 6: Merge the feature branch into master
!git merge feature/robust-input-v2 -m "Merge feature/robust-input-v2: AI-assisted input validation"

# Step 7: Delete the feature branch
!git branch -d feature/robust-input-v2

# Step 8: Show the commit history to verify
!git log --oneline --graph
```

```
Saved working directory and index state On master: AI-assisted modifications for robust input validation
Switched to a new branch 'feature/robust-input-v2'
On branch feature/robust-input-v2
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   todo_list_manager.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .config/
        sample_data/

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (db895453604fa36a080959ed638d76d3ad8b07d)
[feature/robust-input-v2 23be091] AI-assisted: Enhance input validation in main function (re-attempt)
 1 file changed, 20 insertions(+), 6 deletions(-)
Switched to branch 'master'
Updating c1ae8e3..23be091
Fast-forward (no commit created; -m option ignored)
 todo_list_manager.py | 26 ++++++++++++++++++++++------
 1 file changed, 20 insertions(+), 6 deletions(-)
Deleted branch feature/robust-input-v2 (was 23be091).
* 23be091 (HEAD -> master) AI-assisted: Enhance input validation in main function (re-attempt)
* c1ae8e3 Initial commit of To-Do List Manager
```

```
# Read the current content of todo_list_manager.py
with open('todo_list_manager.py', 'r') as f:
    content = f.read()
# Apply AI-assisted modifications for robust input validation
# This involves modifying the main function to handle non-integer choices and empty tasks
# The modified main function
modified_main_function = """def main():
    manager = TodoListManager()
    while True:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task Complete")
        print("4. Exit")
        choice = input("Enter your choice: ")
        # Robust input validation for menu choice
        if not choice.isdigit():
            print("Invalid choice. Please enter a number from 1 to 4.")
            continue
        choice_int = int(choice)
        if choice_int == 1:
            task = input("Enter the task: ")
            if task.strip(): # Prevent adding empty tasks
                manager.add_task(task)
            else:
                print("Task cannot be empty. Please enter a valid task.")
        elif choice_int == 2:
            manager.view_tasks()
        elif choice_int == 3:
            try:
                task_num = int(input("Enter the task number to mark as complete: ")) - 1
                manager.mark_task_complete(task_num)
            except ValueError:
                print("Invalid input. Please enter a number.")
        elif choice_int == 4:
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number from 1 to 4.")
"""
# Replace the original main function with the modified one
# This is a simple string replacement; a more robust solution might use AST parsing for complex changes
import re
updated_content = re.sub(r"def main\(\):\n(?:    .*\n)+if __name__ == '__main__':", modified_main_function + "\nif __name__ == '__main__':", content, flags=re.DOTALL)
# Write the modified content back to the file
with open('todo_list_manager.py', 'w') as f:
    f.write(updated_content)
print("todo_list_manager.py updated with robust input validation.")
```

```
todo_list_manager.py updated with robust input validation.
```

```
!git add todo_list_manager.py
!git commit -m "AI-assisted: Enhance input validation in main function"

On branch feature/robust-input
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .config/
        sample_data/

nothing added to commit but untracked files present (use "git add" to track)
```

```
# Assuming 'feature/robust-input' currently holds the initial commit (c1ae8e3) and the AI-assisted commit (which will be created by f0cd6e3f).
# Step 1: Create a 'master' branch that points to the initial commit (c1ae8e3).
# This establishes 'master' at the base state before AI's changes.
!git branch master c1ae8e3
# Step 2: Checkout 'master'
!git checkout master
# Step 3: Merge 'feature/robust-input' into 'master'.
# This will bring the AI-assisted changes from 'feature/robust-input' into 'master'.
!git merge feature/robust-input -m "Merge feature/robust-input: AI-assisted input validation"
# Step 4: Delete the feature branch
!git branch -d feature/robust-input
# Step 5: Show the log to verify the commit history
!git log --oneline --graph
```

```
M       todo_list_manager.py
Switched to branch 'master'
Already up to date.
Deleted branch feature/robust-input (was c1ae8e3).
* c1ae8e3 (HEAD -> master) Initial commit of To-Do List Manager
```

Explanation:

This task focuses on integrating version control into development using Git. It helps students understand how to manage code changes systematically by working in branches instead of directly modifying the main code. AI can assist by suggesting commands and best practices for commits. Merging changes ensures all updates are combined without conflicts. This reflects real-world collaborative workflows used in software development teams.

Task 3 – AI for Commit Message Generation

• Use AI to generate concise, meaningful commit messages based on the changes made.

• Compare AI-generated messages with human-written ones.

• Discuss how AI can improve documentation and traceability in Git history.

Prompt:

Generate clear and concise Git commit messages based on the code changes made, following best practices.

Code & Output:

```python
# Read the current content of todo_list_manager.py
with open('todo_list_manager.py', 'r') as f:
    content = f.read()
# Define the new delete_task method and the updated main function
new_delete_method = """    def delete_task(self, task_index):
        if 0 <= task_index < len(self.tasks):
            removed_task = self.tasks.pop(task_index)
            print(f"Task '{removed_task['task']}' deleted.")
        else:
            print("Invalid task number.")"""
# Find the last method in the class to insert the new method
# This regex looks for the last 'def' inside the class definition, assuming similar indentation
import re
class_end_match = re.search(r'(\s+def [a-zA-Z_]+\(self,[^:]*\):(?:\n\s{8,}.*)*)(\n\s*def main\(\):)', content, re.DOTALL)
if class_end_match:
    updated_class_content = class_end_match.group(1) + "\n" + new_delete_method + class_end_match.group(2)
    # Replace the class methods part with the new method added
    content_with_new_method = content.replace(class_end_match.group(1) + class_end_match.group(2), updated_class_content)
else:
    content_with_new_method = content # Fallback if regex fails
# The modified main function to include a delete option
modified_main_function_with_delete = """def main():
    manager = TodoListManager()
    while True:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task Complete")
        print("4. Delete Task") # New option
        print("5. Exit")
        choice = input("Enter your choice: ")
        if not choice.isdigit():
            print("Invalid choice. Please enter a number from 1 to 5.")
            continue
        choice_int = int(choice)
        if choice_int == 1:
            task = input("Enter the task: ")
            if task.strip():
                manager.add_task(task)
            else:
                print("Task cannot be empty. Please enter a valid task.")
        elif choice_int == 2:
            manager.view_tasks()
        elif choice_int == 3:
            try:
                task_num = int(input("Enter the task number to mark as complete: ")) - 1
                manager.mark_task_complete(task_num)
            except ValueError:
                print("Invalid input. Please enter a number.")
        elif choice_int == 4: # Handle delete option
            manager.view_tasks()
            if manager.tasks: # Only ask if there are tasks to delete
                try:
                    task_num = int(input("Enter the task number to delete: ")) - 1
                    manager.delete_task(task_num)
                except ValueError:
                    print("Invalid input. Please enter a number.")
        elif choice_int == 5:
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
```

```python
            print("Invalid choice. Please enter a number from 1 to 5.")
"""
# Replace the old main function with the new one including delete option
updated_content = re.sub(r"def main\(\):\n(?:    .*\n)+if __name__ == '__main__':", modified_main_function_with_delete + "\nif __name__ == '__main__':", content_with_new_method, flags=re.DOTALL)
# Write the modified content back to the file
with open('todo_list_manager.py', 'w') as f:
    f.write(updated_content)
print("todo_list_manager.py updated with 'Delete Task' feature.")
```

```
!git add todo_list_manager.py
```

```python
# Import the Python SDK
import google.generativeai as genai
# Used to securely store your API key
from google.colab import userdata

# Configure Gemini API
GOOGLE_API_KEY=userdata.get('GOOGLE_API_KEY')
genai.configure(api_key=GOOGLE_API_KEY)

# Initialize the Gemini API
gemini_model = genai.GenerativeModel('gemini-2.5-flash-preview-04-17')

# Get the diff of the staged changes
import subprocess
def get_git_diff_staged():
    try:
        # --staged shows changes between HEAD and index
        # --unified=0 shows only changed lines, no context lines
        result = subprocess.run(['git', 'diff', '--staged', '--unified=0'], capture_output=True, text=True, check=True)
        return result.stdout
    except subprocess.CalledProcessError as e:
        return f"Error getting git diff: {e.stderr}"

current_diff = get_git_diff_staged()
print("--- Staged Changes Diff ---")
print(current_diff)
print("---------------------------")

if "No changes" in current_diff or not current_diff.strip():
    ai_commit_message = "No staged changes found to generate a commit message."
else:
    # Prompt the AI to generate a commit message
    prompt = f"""Given the following git diff, generate a concise and meaningful commit message (max 70 characters for the subject line, followed by an optional body if needed):\n\n{current_diff}\n\nCommit Message:"""

    print("\nGenerating commit message with AI...")
    response = gemini_model.generate_content(prompt)
    ai_commit_message = response.text.strip()
    print("AI-Generated Commit Message:\n" + ai_commit_message)

# Store the AI-generated message for the next step
# You might want to refine this message before committing
_ai_generated_commit_message = ai_commit_message
```

```python
if '_ai_generated_commit_message' in locals() and _ai_generated_commit_message:
    print(f"Committing with AI-generated message: {_ai_generated_commit_message}")
    !git commit -m "{_ai_generated_commit_message}"
    !git log --oneline --graph
else:
    print("No AI-generated commit message found or it was empty. Please generate one first.")
```

Explanation:

Here, AI is used to automate the process of writing meaningful commit messages. Good commit messages improve readability and make it easier to track project history. By comparing AI-generated messages with human-written ones, students learn what makes a message effective. AI can standardize formatting and ensure clarity. This enhances documentation and improves collaboration in team environments.

Task 4 – Pair Programming Simulation

• Student A writes the initial version of a program.

• Student B, with AI assistance, suggests improvements or adds features.

• Both students collaborate using Git branching and merging to integrate their work.

Prompt:

Simulate pair programming where one developer writes initial code and another improves it with AI suggestions, using Git for collaboration.

Code & Output:

```
def add(x, y):
    return x + y
def subtract(x, y):
    return x - y
def main_calculator():
    print("Select operation:")
    print("1. Add")
    print("2. Subtract")
    while True:
        choice = input("Enter choice(1/2): ")
        if choice in ('1', '2'):
            try:
                num1 = float(input("Enter first number: "))
                num2 = float(input("Enter second number: "))
            except ValueError:
                print("Invalid input. Please enter numbers only.")
                continue
            if choice == '1':
                print(num1, "+", num2, "=", add(num1, num2))
            elif choice == '2':
                print(num1, "-", num2, "=", subtract(num1, num2))
            break
        else:
            print("Invalid input. Please enter 1 or 2.")
if __name__ == '__main__':
    main_calculator()
```

```
•••  Select operation:
     1. Add
     2. Subtract
     Enter choice(1/2): 2
     Enter first number: 64
     Enter second number: 32
     64.0 - 32.0 = 32.0
```

Explanation:

This task mimics real-world pair programming where two developers collaborate on the same project. One focuses on writing the base code, while the other enhances it using AI tools. Git branching allows both contributors to work independently and then merge their work smoothly. This improves teamwork, code quality, and problem-solving skills. It also shows how AI can act as an assistant in collaborative coding environments.